Middleware

🔥 **Hot Tips**

- Middleware hooks run before/after route handlers
- Use for authentication, CORS, logging, or content manipulation
- Pattern matching supports strings (prefix) and regular expressions
- Return `false` from a `before` hook to halt the request

Defining Middleware

Tina4 Ruby uses `before` and `after` hooks — simple blocks that run around your routes.

ruby

```ruby
require "tina4"

# Runs before EVERY request
Tina4.before do |request, response|
  Tina4::Debug.info("Incoming: #{request.method} #{request.path}")
end

# Runs after EVERY request
Tina4.after do |request, response|
  response.headers["X-Powered-By"] = "Tina4 Ruby"
end
```

Pattern Matching

Limit middleware to specific paths using string prefixes or regular expressions.

ruby

```ruby
# Only API routes (string prefix match)
Tina4.before "/api" do |request, response|
  Tina4::Debug.info("API request: #{request.path}")
end

# Regex pattern
Tina4.before /\/admin/ do |request, response|
  # Check admin auth
end

# Multiple API versions
Tina4.after "/api/v2" do |request, response|
  response.headers["X-API-Version"] = "2"
end
```

Halting Requests

Return `false` from a `before` hook to stop the request from reaching the route handler.

ruby

```ruby
Tina4.before "/admin" do |request, response|
  auth_header = request.env["HTTP_AUTHORIZATION"] || ""
  unless auth_header.start_with?("Bearer ")
    response.json({ error: "Unauthorized" }, 401)
    false  # Stops the chain — route handler never runs
  end
end
```

CORS Middleware

ruby

```ruby
Tina4.before do |request, response|
  response.add_cors_headers(
```

```
    origin: "*",
    methods: "GET, POST, PUT, DELETE, OPTIONS",
    headers_list: "Content-Type, Authorization"
  )
end

# Handle OPTIONS preflight
Tina4.options "/{path:path}" do |request, response|
  response.add_cors_headers
  response.status = 204
  response
end
```

Logging Middleware

ruby

```
Tina4.before do |request, response|
  request.env["tina4.start_time"] = Time.now
end

Tina4.after do |request, response|
  start = request.env["tina4.start_time"]
  if start
    duration = ((Time.now - start) * 1000).round(2)
    Tina4::Debug.info("#{request.method} #{request.path} — #{response.status} (
  end
end
```

Execution Order

1. **All matching `before` hooks** run (in registration order)
2. If any returns `false`, the chain stops — route handler is skipped
3. **Route handler** runs

4. **All matching `after` hooks** run (in registration order)

Summary

Feature	Syntax	Notes
Before all	`Tina4.before { |req, res| }`	No pattern = all routes
Before pattern	`Tina4.before "/api" { |req, res| }`	String prefix match
Before regex	`Tina4.before /pattern/ { |req, res| }`	Regex match
After all	`Tina4.after { |req, res| }`	Runs after handler
Halt request	`return false`	Stops chain in `before`

Previous page
Routing

Next page
CSS/SCSS